

Finding the Most Valuable Parts of a Dataset - Group 9

Deniz Cangi
Utrecht University
d.cangi@uu.nl

Bas J. Cornelissen
Utrecht University
b.j.cornelissen@students.uu.nl

Nadin Harshuk
Utrecht University
n.harshuk@students.uu.nl

ABSTRACT

This project explores how to select the most important tuples from a dataset when there is storage constraints. Given a table R , a query workload Q , and a threshold T , we aim to find a subset $R' \subseteq R$ that maximizes the importance of R' based on different importance definitions. Three methods were implemented in PySpark: popularity-based (method 1), popularity-diversity trade-off (method 2), and Shapley-based diversity (method 3). Experiments on synthetic datasets with different distributions showed that popularity-based approach scales best, while diversity-aware approaches yield more representative and diverse subsets at higher computational cost.

1 INTRODUCTION

In a world that keeps becoming more digital, more data is produced and stored every year [1, p. 183] [4, p. 1]. As the global data volume approaches to tens of zettabytes, companies are always looking for ways to get rid of data because they are running out of space to store them or will soon have to face such a situation. This data contains vast amounts of information, but the process of analyzing huge datasets is computationally intensive and it's not sustainable for the future [5, p. 179]. An often chosen approach to improve efficiency is to reduce the size of the dataset [2, p. 1646]. However, this creates a fundamental question: which data should we keep and which should we delete? This project addresses this problem: given a dataset R , a query workload Q , and a threshold T representing the storage limit, we must find a subset of R , R' , where $|R'| = T$ that maximizes importance. But how do we decide which tuples are important?

In this project, we have implemented different approaches to reducing the number of tuples in a dataset, which we discuss in section 2: Solution. We implemented three philosophies for measuring tuple importance: method 1 keeps the most popular tuples (those appearing in the most queries), method 2 keeps popular tuples that are also diverse from each other, and method 3 keeps tuples that maximize overall dataset diversity regardless of queries. Method 1 emphasizes the importance of each tuple independently by ranking tuples only by their query popularity. In method 2, the Jaccard distance between a tuple and the remainder of the subset is incorporated into its importance assessment, balancing popularity with diversity to avoid selecting redundant similar tuples. For this method, we implemented both exact algorithms (subset enumeration that evaluates all possible subsets) and scalable approximations (greedy selection with Locality-Sensitive Hashing), to see the difference between result quality and computational feasibility. method 3 evaluates a tuples importance by measuring the impact of its removal on the subset's diversity using Shapley values. All three methods were implemented

in PySpark on Google Colab and tested on synthetic datasets generated with three different distributions: IID-uniform, clustered, and popular values.

After creating the methods, we conducted an experimental evaluation, which we discuss in section 3: Experimental Evaluation. We tested datasets ranging from 10 to 750,000 tuples. Method 1 achieved the best scalability, processing up to 750,000 tuples in approximately 32 minutes. Method 2's exact subset enumeration finds the globally optimal solution but becomes infeasible beyond 30 tuples, which evaluating all possible subset combinations for $n=30$ and $T=10$ takes 76 minutes. Method 2's greedy versions are practical alternatives to subset enumeration, the exact greedy approach can scale up to 10,000 tuples but requires 95 minutes to compute the subset, while the LSH-based greedy version can process 50,000 tuples in just 62 seconds. Method 3 provides a Shapley-based diversity selection approach, and it has exponential complexity, limiting it to be compatible for small datasets of 25 tuples or fewer (taking 5.5 hours for $n = 25$).

This project is motivated by finding a solution to the practical storage constraints that organizations face daily. Beyond this immediate application of freeing up storage space when databases become too large, these techniques can be applicable to machine learning scenarios as well. In machine learning, training on massive datasets is computationally expensive, which can be an architectural challenge [3, p. 10]. Using diversity-based selection approaches, one can select a diverse and representative subset of the training data that captures important patterns, and train the model with reduced time and cost while maintaining the model quality. However, solving this problem is challenging for several reasons. First, the combinatorial explosion limits exhaustive search, for instance, with method 2 even selecting 10 tuples from 30 tuple dataset requires evaluating more than 30 million 10-tuple combinations, and this requirement grows exponentially with dataset size. Second, there is no ground truth to compare the selected tuples against, which itself requires extensive computation and testing. Third, measuring tuple diversity requires computing pairwise similarity between all tuples, an $O(n^2)$ operation for a dataset with n tuples. That becomes a bottleneck for large datasets. For example, for 10,000 tuples, this means nearly 50 million distance calculations and keeping all these distances in the memory. In terms of implementation, the main challenges we faced were managing memory constraints to avoid crashes and optimizing Spark operations to have efficient parallel processing, though we found that the use of caching, broadcasting, Spark data structures and operations helped to address these issues.

2 SOLUTION

In this section we will describe our solution approach. The assignment stipulated the three methods for us to implement. We still start with explaining our data representation and preprocessing (2.1).

Then we will discuss our approach to the similarity measure, Jaccard Distance (2.2). Following that, we will discuss our approach to method 1 (2.3), method 2 (2.3) and method 3 (2.3).

2.1 Data Representation & Preprocessing

2.1.1 Data Generator. To evaluate the performance and scalability of the proposed methods, a synthetic data generator was implemented. The generator produces integer-based datasets in CSV format, where each row represents a tuple and each column represents an attribute. The generator is fully configurable, allowing customization of the number of rows, number of columns, and the value domains of the attributes. Three main distribution types were supported to simulate different data characteristics:

- (1) **IID-Uniform distribution** – all values occur with equal probability. This distribution provides a baseline for performance under balanced data conditions.
- (2) **Popular value distribution** – a small subset of values appears significantly more frequently than others. This setup allows testing how the methods handle dominant values and popularity biases, which often occur in real-world datasets.
- (3) **Clustered distribution** – values are grouped around specific ranges or centers, imitating real data where similar entities tend to form clusters. This enables testing diversity sensitivity and how each method performs when data points are densely packed in specific regions.

These distribution options enable testing under both idealized and realistic conditions, allowing the study of how each method reacts to varying levels of value repetition, data skewness, and diversity. The generated datasets were later used as input to the three subset-selection methods, with different dataset sizes used to assess runtime scalability.

2.1.2 Query Generator. In addition to the datasets, a query generator was developed to simulate realistic user queries over the generated data. The generator allows customization of the number of queries and the number of conditions per query, where each condition corresponds to a pair (attribute, value) that must be matched by tuples in the dataset. To reflect real-world query behavior, approximately 70% of the query conditions were designed to reference popular values (the most frequent attribute values in the dataset). This design choice ensures that certain tuples appear in multiple queries, producing realistic popularity scores that mimic how users often search for or interact with similar data points repeatedly. Each query record is stored with:

- **Query ID** – a unique identifier for each query.
- **Conditions list** – stored in JSON format.
- **Condition count** – the number of predicates in the query.

This structure makes it easy to parse data and expand it efficiently in Spark during later stages of computation.

2.2 Similarity Measure: Jaccard Distance

For method 2 and 3, a method was needed to calculate the similarity between two tuples. For this, we have used Jaccard Distance. Our rationale here was the following. Our synthetic datasets use integers as categorical values (not numeric quantities with meaningful ordering). Jaccard treats each integer as a distinct category, measuring

set overlap rather than numeric distance. Using it is an efficient computation and works well with complex datasets. The definition of Jaccard similarity (of Jaccard index) is as follows:

$$\text{sim}(A, B) = \frac{A \cap B}{A \cup B}$$

Jaccard distance is then:

$$\text{JaccardDistance}(A, B) = 1 - \text{sim}(A, B)$$

Our program does the following to compute the similarity of tuples. First it counts the number of columns where the tuples have the same value. Secondly, it counts the union of the tuples by counting the columns. It then calculates the Jaccard Similarity, by dividing the former with the latter. Finally, we calculate the Jaccard Distance by subtracting this value from 1. For the exact implementation, please see the `dissimilarity()` function in our code files.

2.3 Method 1

Algorithm 1: Method 1: Popularity-Based Selection

Input: Table R , query set Q , threshold T
Output: $R' \subseteq R$, $|R'| \leq T$, and $\text{imp}(R')$
Assign stable `row_id` to all $t \in R$; cache R ;
Initialize $\text{pop}(t) \leftarrow 0$ for all $t \in R$;
foreach $q \in Q$ **do**
 $\text{Ans}(q) \leftarrow \{t \in R : t \text{ satisfies all } A=v \text{ in } q\}$;
 foreach $t \in \text{Ans}(q)$ **do**
 $\text{pop}(t) \leftarrow \text{pop}(t) + 1$
 $\text{TOTAL_POP} \leftarrow \sum_{t \in R} \text{pop}(t)$;
if $\text{TOTAL_POP} = 0$ **then**
 return $R' = \emptyset$, $\text{imp}(R') = 0$
Sort R by ($\text{pop}(t)$ desc, `row_id` asc);
 $R' \leftarrow$ first T tuples of the sorted list;
 $\text{imp}(R') \leftarrow \frac{\sum_{t \in R'} \text{pop}(t)}{\sum_{t \in R} \text{pop}(t)}$;
return R' , $\text{imp}(R')$;

In method 1, the approach to the problem was to simply use the amount of queries a tuple was selected as the definition of its importance. We call this Popularity-Based Selection.

Tuple importance can be expressed as follows:

$$\text{imp}(t) = \text{pop}(t) = \text{number of queries containing } t$$

The importance of a subset of tuples (R'), is then calculated simply as the sum of the importance of the tuples in the set. When we divide this subset with the set of all tuples (R), we find a normalized result importance value for the subset.

$$\text{imp}(R') = \frac{\sum_{i=1}^{|R'|} \text{pop}(t_i) \text{ for } t_i \in R'}{\sum_{i=1}^{|R|} \text{pop}(t_i) \text{ for } t_i \in R}$$

The method starts by loading both the tuple set R and the query table Q as Spark DataFrames and given unique IDs for distributed processing. The query log has a `predicates_json` column that lists each query's conditions. This column is converted to a key-value structure using Spark's `MapType`, so each query associates attributes with their target values. Then every query is divided into multiple rows using the `explode()` function, resulting in a table structured like `(query_id, col, value, k)`, where k represents the number of conditions in the query. For the dataset, all row values are compressed into a map of column-value pairs `(row_id, map(col, value))` rather than maintaining the full table format.

The method then joins the queries and data tables on `col_value_pairs[F.col("col")] == F.col("value")`.

This produces a table `(query_id, k, row_id)` containing each tuple that satisfies at least one condition of the query.

After that, the method retains the tuples that satisfy all conditions of a query by counting the number of matching conditions for each `(query_id, row_id)` pair and checking whether the count equals the query's k conditions, using `groupBy`, aggregation, and filtering:

```
groupBy("query_id", "row_id").agg(count("*").alias("match_count"))
```

After this step, the frequency of each `row_id` in the table is counted to determine its popularity score.

```
popularity_df = all_matches.groupBy("row_id").agg(countDistinct("query_id").alias as ("popularity"))
```

The tuples are then ordered by their popularity in descending order, and the top T most popular tuples are selected. This method relies heavily on Spark's column-based operations such as `JOIN`, `groupBy()` and `count()`, which ensure efficiency even for large datasets. The main computational cost stems from the join and aggregation stages, estimated at roughly $O(|R| \times |Q| \times k)$. By caching intermediate results, the method achieves good scalability and serves as a solid reference baseline for the following approaches.

2.4 Method 2

The second method builds on method 1 by adding tuple dissimilarity as a weighted factor in the importance calculation.

$$imp(t) = pop(t) \times \frac{\sum_{i=1}^{|R'| - \{t\}} (1 - sim(t, t_i))}{|R'|}$$

$$imp(R') = \frac{\sum_{i=1}^{|R'|} imp(t_i)}{\sum_{i=1}^{|R|} imp(t_i)} = \frac{\sum_{i=1}^{|R'|} pop(t_i) \times \frac{\sum_{k=1}^{|R'| - \{t_i\}} (1 - sim(t_i, t_k))}{|R'|}}{\sum_{i=1}^{|R|} pop(t_i) \times \frac{\sum_{k=1}^{|R| - \{t_i\}} (1 - sim(t_i, t_k))}{|R|}}$$

2.4.1 Exact Subset Enumeration. The implementation goes through several steps: first, it calculates popularity, then it filters out tuples with $popularity = 0$, then it precomputes the necessary pairwise dissimilarities, and finally, it uses these values along with popularity scores to evaluate all subset combinations of size T , and by the end the subset with the highest score is chosen.

Initially, the popularity of tuples in dataset R is computed using the above discussed method 1. Tuples with $pop(t) = 0$ are filtered

Algorithm 2: Method 2: Exact Exhaustive Subset Enumeration

Input: Table R , query set Q , threshold T

Output: $R' \subseteq R$, $|R'| = T$, and $imp(R')$

Assign stable `row_id` to all $t \in R$; cache R ;

Compute $pop(t)$ for all $t \in R$ using Method 1;

$C \leftarrow \{t \in R : pop(t) > 0\}$;

if $C = \emptyset$ **or** $T \leq 0$ **then**

return $R' = \emptyset$, $imp(R') = 0$

if $T > |C|$ **then**

$T \leftarrow |C|$

Compute exact pairwise distances $d(t, u)$ for all $t, u \in C$ (Jaccard distance);

Broadcast $pop(\cdot)$ and $d(\cdot, \cdot)$ to all workers;

$denom \leftarrow \sum_{t \in C} pop(t) \cdot \frac{1}{|C| - 1} \sum_{u \in C \setminus \{t\}} d(t, u)$;

$bestSubset \leftarrow \emptyset$, $bestNum \leftarrow -\infty$;

foreach $S \subseteq C$ with $|S| = T$ (in batches) **do**

$num(S) \leftarrow \sum_{t \in S} pop(t) \cdot \frac{1}{|S| - 1} \sum_{u \in S \setminus \{t\}} d(t, u)$;

if $num(S) > bestNum$ **then**

$bestNum \leftarrow num(S)$;

$bestSubset \leftarrow S$;

$R' \leftarrow bestSubset$;

$imp(R') \leftarrow bestNum / denom$;

return R' , $imp(R')$;

out, resulting in a candidate pool. This step ensures that no time is wasted on tuples that don't contribute to identifying the most important tuples or subsets.

Jaccard pairwise distances (see 2.2) are computed within the candidate pool. These computations are subsequently reused in the subset-selection procedure and in the denominator calculation during normalized importance calculation.

To enable efficient pairwise distance lookup, several preparations are implemented after computing the initial popularity scores and filtering the tuples. First, a symmetric version of the distance matrix is generated to avoid redundant distance computations. Subsequently, an adjacency map (RDD) is constructed, mapping each tuple to a list of its neighboring tuples and the corresponding distances: `tuple_ID  $\rightarrow$  [list of [neighbor_ID, distance]]`. This structure allows each worker to efficiently retrieve the required data and helps avoid extensive `JOIN` and `FILTER` operations and multiple scans across the entire table. The RDD is partitioned and stored both in memory and on disk.

Both the adjacency and popularity maps from method 1 are broadcasted to the Spark workers. Broadcasting ensures data availability across all workers, eliminating the need for repeated shuffles or joins during subset evaluations.

The next step involves calculating the overall importance of the dataset R (the denominator) using the `compute_denominator_exact()` method, which is calculated once.

The `generate_combinations(candidate_ids, T)` method is then used to generate all possible sub-datasets of size

T . To prevent memory overflow, data are processed in batches of 10,000. Each batch is parallelized across Spark workers, with each worker processing a subset of combinations.

Activating the `compute_numerator_for_subset` (subset) method, which, for each subset, retrieves the precomputed pairwise distances from broadcasted variables and calculates the total importance of the subset as the sum of per-tuple contributions (*popularity* $\times$ *average distance to others in the subset*). The subset with the highest importance score for each batch is compared to the current global best. And after all batches are processed, the subset with the overall highest importance is returned as R' .

Finally, the normalized importance score is calculated by dividing the selected subset's importance score by the previously calculated denominator.

The exhaustive approach runs in $O(C(n, T)T^2)$ time, guaranteeing to find the subset with best importance score but remaining feasible only for small datasets or low T values. Beyond the exhaustive method described above, two heuristic variants were implemented to assess trade-offs between efficiency and accuracy.

2.4.2 Exact Greedy Selection. This variant resembles the Exact Subset Enumeration solution. It uses the exact Jaccard distances between the tuples to assess their diversity. So the first parts of the implementation are the same as before. The primary difference is, when creating possible sub-datasets of size T , a greedy algorithm is used instead of generating all possible combinations as in the Exact Subset Enumeration solution. Also, because Spark is optimized for parallel data transformation rather than step-by-step loops, the greedy loop was implemented using Python loops instead. The algorithm begins by initializing tracking structures:

- (1) the remaining set of all candidate IDs.
- (2) A scores with id: 0.0 for all candidates to track diversity.
- (3) B scores with id: 0.0 for all candidates to track contribution to others.
- (4) Distance cache to store distances for selected tuples.

During the first iteration ($i = 0$), the algorithm selects the tuple with the highest popularity score, adds it to `selected_ids`, and removes it from `remaining`.

For the remaining iterations ($i = 1$ to $T - 1$), for each remaining tuple, a score is calculated which includes the benefit (`B_scores`) and importance (`pop` $\times$ `A_scores`) of the tuple.

```
Compute score(s) = B_scores[s] + pop(s)  $\times$  A_scores[s]
```

After each selection, the algorithm updates the diversity and contribution scores for the remaining candidates. For each remaining tuple, it computes and caches the distance to the newly selected tuple.

```
A_scores[s] += dist
```

```
B_scores[s] += pop(best_id)  $\times$  dist
```

This step ensures that future selections take into account the newly added tuple's influence on both diversity and popularity. Once there are T tuples in the selected set, a final importance calculation is made, using the cached distances and popularity scores to obtain each tuple's importance score.

These scores are then used to calculate the numerator, and the previously computed denominator is used to obtain the final importance score of the subset T .

The exact greedy approach operates in $O(n^2 + T \times n)$ time. It is a heuristic solution that is faster and requires less computational power, making the approach effective for medium-sized datasets and higher thresholds.

2.4.3 LSH-Based Greedy Selection. This variant resembles the Exact Greedy Selection solution. The primary difference is that, instead of calculating all pairwise distances, LSH is used to identify tuples that are likely to be similar.

This variant begins the same way as the other ones, using method 1 to calculate the popularity score of the tuples and filtering all tuples that score 0.

After that, each row in the table was converted into a sparse vector for the LSH algorithms to process them more easily. To do that, the rows were turned into text tokens in the shape of `["col: value" . . .]`, and those tokens were then converted into binary feature vectors by applying the `hashingTF` function.

With the tuples having the right structure, we fit `MinHashLSH` to the feature vectors, which creates hash functions for fast similarity search by generating hash signatures for each row; those signatures let the approximation distance algorithms know which rows are similar.

LSH is then applied to estimate the Jaccard distance by comparing pairs of rows that have landed in the same or nearby hash buckets. The Jaccard distance is estimated by comparing their signatures, and distances above 0.8 are filtered out.

The near pairs are then stored in the format `[tuple1_id, tuple2_id, dissimilarity_score]`. With these approximate distances and the previously calculated popularity scores, the greedy selection loop is executed, following the same procedure as the Exact Greedy algorithm. The process iteratively builds the subset of size T by evaluating which tuple contributes most to the importance score of the subset, starting with the most popular tuple and continuing until the subset reaches size T .

After the subset is chosen, the numerator is computed as the average distance multiplied by the popularity, and the denominator is calculated as the average distance multiplied by the overall popularity. The final importance score is obtained by dividing the numerator by the denominator.

The LSH-Based Greedy approach operates in $O(n^2 + T \times n)$ time. It is a heuristic solution that reduces memory and runtime consumption, enabling the processing of datasets with hundreds of thousands of rows by using approximate pairwise distance calculations and greedy selection to choose the top T tuples rather than generating all possible combinations. Although this method is approximate and cannot guaranty exact results, it achieves scalability beyond that of the exact solution.

2.5 Method 3

In method 3, the importance of a set of tuples is determined by the similarity they had to their centroid. The more heterogeneous the set, the higher its importance. The importance of a single tuple is then determined by how much it changes the overall importance of the set when it is included or removed, which is called residual importance.

We define the importance of a set of tuples as the average distance from the tuples to the centroid of the set. For all tuples in the set R' , we sum the distance from tuple to the centroid of R' and divide it

Algorithm 3: Method 3: Residual-Importance Selection**Input:** Table R , threshold T **Output:** $R' \subseteq R$, $|R'| = T$, and $\text{imp}(R')$ Assign stable `row_id` to all $t \in R$; cache R ;Extract all tuples of R into memory as (`row_id` $\rightarrow$ values);**foreach** tuple $t \in R$ **do** Compute $\text{resimp}(t)$ using Shapley-style averaging;; For all subsets $S \subseteq R \setminus \{t\}$ (processed in batches and parallelized): (1) $\text{imp}(S) \leftarrow$ average distance of tuples in S from the centroid of S ; (2) $\text{imp}(S \cup \{t\}) \leftarrow$ average distance of $S \cup \{t\}$ from its centroid (updated incrementally); (3) $\Delta \leftarrow \text{imp}(S \cup \{t\}) - \text{imp}(S)$; (4) Weight Δ by $\frac{|S|! \cdot (|R| - |S| - 1)!}{|R|!}$ and add to $\text{resimp}(t)$;Rank all tuples by $\text{resimp}(t)$ in descending order; $R' \leftarrow$ top T tuples by $\text{resimp}(t)$; $\text{imp}(R') \leftarrow$ average distance of tuples in R' from the centroid of R' ;**return** R' , $\text{imp}(R')$;

by the cardinality of R . The centroid is the tuple that best represents the set. We calculate this by going over each column, calculating its centroid by using mode (the most frequent value), since our data is integer based and it acts as categorical values. As a result, the more diverse/heterogeneous the set is, the higher its importance will be.

To calculate the importance of a single tuple in the set, we use Shapley value measuring contribution, by calculating the residual importance of the set when the tuple is taken out. To get the residual importance of the tuple t we sum up all the marginal contributions of adding t to the subset multiplied by a Shapley weight:

$$\text{resimp}(t) = \sum_{S \subseteq R - \{t\}} \text{weight} * \text{marginal contribution of } t \text{ to } S$$

$$\text{weight} = \frac{|S|! \cdot (|R| - |S| - 1)!}{|R|!}$$

$$\text{marginal contribution} = \text{imp}(S \cup \{t\}) - \text{imp}(S)$$

In short, the idea here is that tuples that increase diversity of many subsets are valuable.

The main steps behind our algorithmic implementation is as follows:

- Consider all possible subsets S that don't contain tuple t .
- For each subset S :
 - How diverse is S by itself?
 - How diverse is S after adding t ?
 - Difference between these value's is t 's contribution to S .
- Weight each contribution by the Shapley formula to ensure fairness across different subset sizes
- Average across all subsets equals t 's residual importance

A challenge with this implementation is the increase in amount of possible subsets when the number of tuples in R increase. From

a set of n tuples, we create and evaluate 2^n subsets. When $n = 20$ means 1,048,576 subsets computations and when $n = 25$ this already means 33,554,432 subsets. Since we have limited resources, we cannot compute all the subsets at once, as this results in memory overflow. As a solution, we have implemented the algorithm using Batch processing and Spark parallelization.

2.5.1 Calculating Residual Importance of the Tuples. We have worked out our implementation in more detail below.

(1) Preparation

(a) Broadcast data to spark workers

(b) Compute `factorial(i)` for $i = 0$ to n and store them in array for $O(1)$ lookup when calculating Shapley weights

(2) Level by level subset enumeration

(a) Process subsets by size $k = 1, 2, 3, \dots, n - 1$

Why level-by-level?

- For memory, we can release $k - 1$ subsets before processing k
- Allows easy tracking of the progress
- Parallelization, as this way, each level is independent

(3) For each level k :

(a) Generate Subsets in Batches:

Generate subsets of size k from data in batches to avoid memory overflow: we avoid loading all $C(n, k)$ combinations in memory at once. Instead we do computations in batches of 10000 combinations.

(b) Distribute batch to spark workers

Parallelize batch across partitions

(c) We implemented a worker function that compute contributions of the tuples for the current batch

• For each subset S in batch:

(i) Skip empty subsets

(ii) compute centroid, importance and Shapley weight of S (d) For each tuple t NOT in S :

(i) Do incremental centroid update to $S \cup \{t\}$ when we add t to S (incremental centroid update is one of the optimizations we did to avoid recalculating the centroid. Explained below in detail)

(ii) Compute importance of $S \cup \{t\}$ (iii) Marginal contribution of t to $S = \text{imp}(S \cup \{t\}) - \text{imp}(S)$ (iv) $\text{contribution} = \text{weight} \cdot \text{marginal}$ (4) Get residual importance results for the tuples, select top T tuples as selected subset

(5) Compute diversity of the selected set and return

It is important to note at 3d, that the contributions are calculated with the MapReduce paradigm. We use map reduce to gather all contributions corresponding to that tuple. In the map phase, each worker processes its subsets and residual importance of tuples to that subset. In the reduce phase, the contributions are aggregated by tuple id.

As stated earlier, we computed the centroid by for each column counting the occurrences of the values and selecting the most common. When we compute this centroid value for a subset, we also store the count of each value, so that we can easily compute the incremental centroid for the subset with the tuple added. The computational complexity are as follows:

- Recompute from scratch $O(|S| \times D)$
- Recompute with additional tuple with results from previous calculation: $O(D)$

The overall subset R' importance was computed by calculating the centroid of the subset first. Then for each tuple in the subset the distance between the tuple and the centroid was calculated, these were all summed together and finally divided by the size of the subset.

The complexity of method 3 is as follows:

- Time: $O(2^n \times n \times D)$, where D stands for distance.

We can conclude that it is exponential in n , which we will analyze more in section 3.3.

3 EXPERIMENTAL EVALUATION

3.1 Method 1 Results

Method 1's implementation takes the query table and the dataset and restructures them in `[query_id, k, col, value]` and `[tuple_id, map[col, value]]` to perform joins on their `[column, value]` pairs. It then filters out all tuples that do not satisfy all k conditions of the query. The occurrences of `[query_id, tuple_id]` per `[tuple_id]` are counted to determine the popularity score of each tuple, and the top subset of size T is selected based on these scores.

Given that method 1 has a time complexity of $O(|R| \times |Q| \times k)$, it is capable of handling large dataset sizes.

We evaluated method 1 on datasets containing between 100,000 and 500,000 rows, in increments of 100,000, with each dataset containing 10 columns and each query log containing 1,000 queries. Three types of datasets were tested: clustered, uniform, and popular values, each using a 15% threshold.

Table 1: Method 1 Runtime and Coverage on IID-Uniform Dataset (10 columns, 1000 queries)

Rows (n)	Threshold (T)	Coverage (%)	Runtime (s)
100,000	15,000	1.09	49.08
200,000	30,000	0.83	113.55
300,000	45,000	0.70	345.19
400,000	60,000	0.47	569.00
500,000	74,000	0.38	753.98
750,000	112,500	0.25	1906.54

Table 1 presents the results for method 1 across these dataset sizes (on IID-Uniform datatype), illustrating how runtime and performance scale with increasing data volume. As expected, runtime increases with each additional 100,000 rows. However, coverage (*importance score* $\times 100$) decreases as dataset size grows. This decline occurs because the denominator, defined as the sum of all R popularity scores, increases more rapidly than the numerator, which is the popularity score of R' . Although the threshold also increases, it is insufficient to maintain coverage at larger scales.

Table 2 presents results for method 1 across different data types, illustrating how data type influences the outcomes.

A clear difference is observed between the data types.

Table 2: Method 1 Performance Across Dataset Distributions and Sizes (10 columns, 1000 queries)

Dataset Type	Rows (n)	Coverage (%)	Runtime (s)
IID-Uniform	100,000	1.09	49.08
IID-Uniform	200,000	0.83	113.55
IID-Uniform	300,000	0.70	345.19
IID-Uniform	400,000	0.47	569.00
IID-Uniform	500,000	0.38	753.98
IID-Uniform	750,000	0.25	1906.54
Clustered	100,000	0.74	89.30
Clustered	200,000	0.44	186.29
Clustered	300,000	0.16	674.02
Clustered	400,000	0.13	868.44
Popular Values	100,000	0.26	264.63
Popular Values	200,000	0.15	661.35

- (1) Dataset Size - IID-Uniform type reached up to 750k without crashing, 400k for clustered, and 200k for popular values.
- (2) Runtime - for the runtime, we can see that the IID-Uniform type was the fastest, followed by Clustered, which was 200% slower, and then Popular-Values, which was 500% slower.
- (3) Coverage (popularity score multiplied by 100) - The IID-Uniform type maintained the highest coverage at 0.25 for 750,000 rows, compared to the clustered type at 0.13 for 400,000 rows and the popular values type at 0.15 for 200,000 rows.

For the IID-Uniform type, method 1 processed up to 750,000 rows while maintaining the highest coverage and fastest runtime compared to other data types. This performance is influenced by the number of tuples that satisfy the k conditions in each query. In datasets with popular values or clustered distributions, repeated values increase the likelihood that many tuples match query conditions. As a result, more tuples are involved in join and aggregation operations, leading to larger operations in Spark, increased memory usage, and longer runtimes. Additionally, repeated values tend to receive higher popularity scores, which increases the denominator (the sum of popularity scores for all tuples in R), causing the overall popularity score to grow significantly. In contrast, the IID-Uniform distribution lacks such bias; tuples are randomly and evenly distributed. This results in fewer join and aggregation operations, as tuples are less likely to be similar or have repeating values.

3.2 Method 2 Results

3.2.1 Exact Subset Enumeration. Our primary implementation for method 2 uses exact subset enumeration, which exhaustively evaluates all possible subsets of size T from the dataset to find the subset with maximum importance. This approach guarantees finding the optimal solution since we check every possible combination for the given threshold. The algorithm generates all $\binom{n}{T}$ combinations, where n is the number of candidate tuples (those with *popularity* > 0) and T is the threshold. For each combination, we compute its importance by calculating the popularity-weighted average dissimilarity for each tuple in the subset, summing these values, and dividing by the total importance of all data. The time complexity is $O(\binom{n}{T} \times T^2)$, where

the T^2 comes from computing pairwise distances between tuples in each subset.

To evaluate this approach, we ran experiments on IID-uniform distributed, popular values and clustered datasets with 5 columns and 1,000 queries. In this section, we focus on results obtained from the IID-uniform distribution. In our experiments, we systematically varied two key parameters: the dataset size (n) and the threshold (T). We tested dataset sizes from 10 to 100 rows, and used thresholds of 5 and 10. All experiments were conducted on Google Colab using PySpark with batch processing (batch size of 10,000 subsets) to handle memory constraints. The goal was to understand how the algorithm scales with both dataset size and threshold, and to identify practical limits for when exact enumeration remains feasible.

The number of subsets we must evaluate is determined by the binomial coefficient $\binom{n}{T} = \frac{n!}{T!(n-T)!}$. This grows extremely rapidly as either n or T increases. For example, with $n = 25$ rows and $T = 5$, we evaluate $\binom{25}{5} = 53,130$ combinations. However, increasing the threshold to $T = 10$ on the same dataset yields $\binom{25}{10} = 3,268,760$ combinations which over 60 times more. This combinatorial explosion is the fundamental bottleneck of the exact approach.

Table 3 presents our complete runtime results. The most important observation is the impact of the threshold parameter. For a dataset with 25 rows, increasing T from 5 to 10 causes runtime to jump from 18.17 seconds to 518.45 seconds (8.6 minutes) which is a 28.5 $\times$ increase. The situation becomes even more severe at 30 rows: runtime increases from 52.22 seconds at $T = 5$ to 4,608.03 seconds (76.8 minutes) at $T = 10$, which is an 88 $\times$ increase. While the combination count increases by 210 $\times$ in this case, our parallel Spark implementation provides some efficiency gains, though the exponential trend is unavoidable.

Table 3: Method 2 Exact Enumeration Runtime Performance on IID-Uniform Distribution

Rows (n)	Cols	Threshold (T)	Combinations	Runtime (sec)
10	5	5	252	11.43
15	5	5	3,003	10.78
20	5	5	15,504	13.49
20	5	10	184,756	56.34
25	5	5	53,130	18.17
25	5	10	3,268,760	518.45
30	5	5	142,506	52.22
30	5	10	30,045,015	4,608.03
35	5	5	324,632	63.47
40	5	5	658,008	113.79
50	5	5	2,118,760	329.01
100	5	5	75,287,520	11,315.58

The impact of dataset size is equally concerning when keeping the threshold fixed. Table 4 isolates this factor by showing results for $T = 5$ across different dataset sizes. When we double the dataset row counts from 50 to 100 rows, there is a 35.5 $\times$ increase in combinations and a 34.4 $\times$ increase in runtime. The 100-row experiment took over 3 hours (188.59 minutes) to complete, demonstrating that even with a relatively small threshold of 5, exact enumeration quickly becomes impractical. The runtime growth is roughly linear with the combination count, which makes sense since each subset requires approximately the same amount of computation.

Table 4: Runtime Growth with Fixed Threshold ($T = 5$) on IID-Uniform Distribution

Rows (n)	Combinations	Runtime (sec)	Runtime (min)
10	252	11.43	0.19
25	53,130	18.17	0.30
50	2,118,760	329.01	5.48
100	75,287,520	11,315.58	188.59

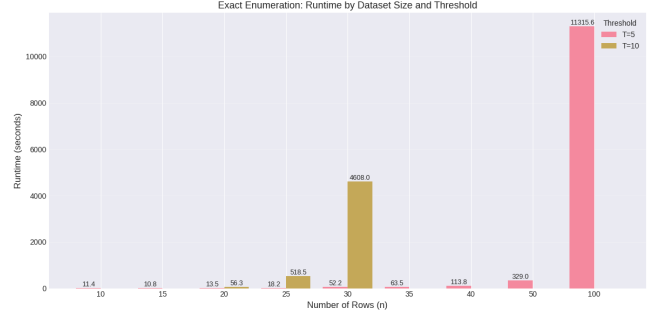


Figure 1: Runtime comparison for Method 2 exact enumeration across different dataset sizes and thresholds on IID-uniform distribution. The increase in runtime with threshold is evident.

Figure 1 visualizes the runtime behavior by dataset size and threshold. The bar chart shows the exponential growth pattern, with the $T = 10$ cases (gold bars) towering over the $T = 5$ cases (pink bars) for the same dataset sizes. The 100-row, $T = 5$ experiment stands out as taking substantially longer than all other $T = 5$ experiments, highlighting how quickly this approach becomes infeasible.

Based on these experiments, we can categorize the practical limits of exact enumeration. For datasets with $n \leq 30$ rows and $T \leq 5$, the algorithm finishes in under one minute. For slightly larger problems with $n \leq 50$ rows and $T \leq 10$, runtimes range from minutes to approximately two hours, which is slow but still manageable for offline processing. However, for $n > 50$ or $T > 10$, runtimes extend to many hours and even days, making exact enumeration impractical for real-world applications. It is worth noting that these experiments were conducted on Google Colab with limited resources (1-2 Spark workers). An environment with hundreds or thousands of workers would provide substantially better performance through increased parallelization. However, even with massive computational resources, the combinatorial growth makes it hard to scale to realistic dataset sizes with thousands or millions of tuples.

3.2.2 Greedy Algorithms. To address the scalability limitations of exact enumeration, we implemented two greedy approximation algorithms. Both use the same selection strategy, they iteratively pick the tuple that maximizes the marginal gain in subset importance. The key difference lies in how they compute distances between tuples. The Exact-Greedy algorithm computes exact Jaccard distances for all pairs of tuples and stores them in a distributed adjacency map for fast lookup during selection. The LSH-Greedy algorithm uses MinHash Locality-Sensitive Hashing to approximate distances, only storing pairs that are within a threshold distance, and assumes distant pairs

have distance 1.0. The theoretical complexity is $O(n^2 + T \times n)$ for Exact-Greedy, where the n^2 term comes from computing all pairwise distances and the $T \times n$ term comes from the greedy selection loop. For LSH-Greedy, the complexity is roughly $O(n \times k_{LSH} + T \times n)$, where k_{LSH} is the average number of near neighbors per tuple found by LSH, which is typically smaller than n .

We evaluated both greedy algorithms on the IID-uniform distribution, popular values and clustered datasets. In this section, we focus on results obtained from the IID-uniform distribution. We tested both algorithms with testing datasets ($n \leq 100$ with $T = 5$) to compare with Exact Enumeration algorithm and large datasets (up to 50,000 rows with $T = 1000$) to assess their scalability.

Table 5 shows the complete runtime results. For small datasets, both algorithms perform similarly with runtimes under 15 seconds. The results are noisy at this scale because Spark overhead dominates the actual computation time. However, the difference becomes significant at larger scales. At 1,000 rows with $T = 100$, Exact-Greedy takes 86.01 seconds while LSH-Greedy takes 31.69 seconds which is a 2.7 $\times$ speedup. At 5,000 rows with $T = 500$, Exact-Greedy requires 1,092.30 seconds (18.2 minutes) compared to LSH-Greedy’s 26.43 seconds, which is a 41 $\times$ speedup. At 10,000 rows with $T = 1000$, Exact-Greedy takes 5,690.30 seconds (94.8 minutes) while LSH-Greedy completes in just 29.05 seconds, a 196 $\times$ speedup. We attempted to run Exact-Greedy on 50,000 rows but it crashed due to memory constraints, while LSH-Greedy completed it successfully in 62.09 seconds.

Table 5: Total Runtime Comparison: Greedy Algorithms on IID-Uniform Distribution

Rows (n)	Exact-Greedy (s)	LSH-Greedy (s)
10	9.11	7.17
20	8.29	4.72
50	7.41	6.83
100	11.40	9.45
1,000	86.01	31.69
5,000	1,092.30	26.43
10,000	5,690.30	29.05
50,000	<i>crashed</i>	62.09

The runtime behavior differs between the two approaches at larger scales. For Exact-Greedy, runtime increases rapidly from 11.40 seconds at $n = 100$ to 86.01 seconds at $n = 1,000$, then jumps to 1,092.30 seconds at $n = 5,000$ and 5,690.30 seconds at $n = 10,000$. This steep growth can be explained by the $O(n^2 + T \times n)$ complexity, where both n and T are increasing together in our experiments. As both parameters grow proportionally, the quadratic distance computation and the linear greedy selection both contribute to the increasing runtime. In contrast, LSH-Greedy maintains remarkably stable performance, with runtimes between 26 and 32 seconds across all large datasets (1,000 to 10,000 rows), only increasing slightly to 62 seconds at 50,000 rows. This is because LSH avoids computing all pairwise distances, instead only processing and keeping only the nearby pairs, which keeps the precomputation cost low and making greedy selection easier regardless of dataset size.

Table 6 shows how the precomputation phase scales. For Exact-Greedy, precompute time grows from 0.49 seconds at 100 rows to

65.51 seconds at 10,000 rows, following the expected quadratic pattern. In contrast, LSH-Greedy’s precompute time remains relatively flat (between 2.82 and 5.81 seconds) across all dataset sizes.

Table 6: Precomputation Runtime: Distance Computation Phase

Rows (n)	Exact-Greedy (s)	LSH-Greedy (s)
100	0.49	0.89
1,000	6.18	5.69
5,000	14.13	2.82
10,000	65.51	4.52
50,000	<i>crashed</i>	5.81

Figure 2 visualizes the scalability difference. The Exact-Greedy curve shows steep quadratic growth that would continue worsening at larger scales, while the LSH-Greedy line remains nearly flat, demonstrating practical scalability to real-world dataset sizes. It is important to note that both greedy algorithms are only *approximations* to the true optimization objective defined by method 2. Because they make local, step-by-step decisions instead of evaluating all possible subsets, they do not guarantee finding the globally optimal set of tuples. Moreover, LSH-Greedy introduces an additional layer of approximation by estimating pairwise distances rather than computing them exactly. Therefore, these methods do not produce the 100% correct results that Exact Subset enumeration would yield especially in the larger datasets. However, on small datasets where exact enumeration was feasible, the greedy methods selected identical tuples with few tweaks, suggesting that the loss in accuracy is minor compared to the runtime gains. Therefore, if small deviations from the optimal importance can be tolerated, the substantial improvement in scalability makes these greedy approaches highly practical.

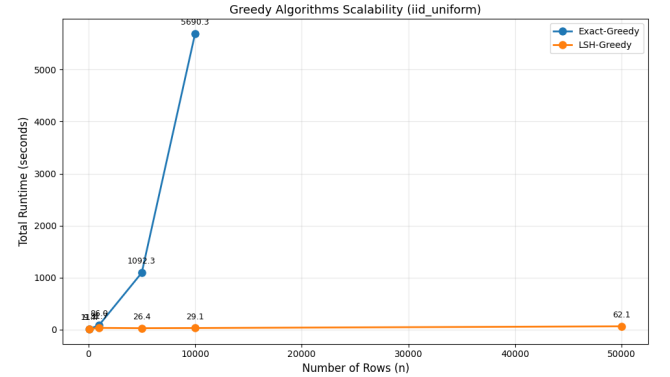


Figure 2: Scalability comparison between Exact-Greedy and LSH-Greedy algorithms on IID-uniform distribution. LSH-Greedy maintains near-constant runtime while Exact-Greedy shows quadratic growth and crashes at 50,000 rows.

3.2.3 Performance Across Data Distributions. To understand how different data distributions affect algorithm performance, we compared all three algorithms on three data distributions: IID-uniform, clustered, and popular values. We selected representative test cases for each algorithm based on their practical operating range, for exact

enumeration with $n = 50, T = 5$; exact greedy with $n = 5,000, T = 500$; and LSH-greedy with $n = 10,000, T = 1,000$. Table 7 summarizes the results.

Table 7: Algorithm Performance Across Data Distributions

Distribution	Algorithm	Rows	T	Score	Runtime (s)
clustered	Exact-Enum	50	5	0.339	329.90
iid_uniform	Exact-Enum	50	5	0.160	329.01
popular_values	Exact-Enum	50	5	0.230	330.71
clustered	Exact-Greedy	5,000	500	0.208	936.85
iid_uniform	Exact-Greedy	5,000	500	0.205	1,092.30
popular_values	Exact-Greedy	5,000	500	0.158	973.90
clustered	LSH-Greedy	10,000	1,000	0.215	64.79
iid_uniform	LSH-Greedy	10,000	1,000	0.210	29.05
popular_values	LSH-Greedy	10,000	1,000	0.139	208.28

The clustered distribution dataset achieves the highest importance scores across all three algorithms. This makes sense because when generating clustered datasets we divided rows into clusters where tuples within each cluster have similar values. When the selection algorithm picks tuples for the subset R' , it tends to choose representatives from different clusters to maximize diversity, resulting in higher average dissimilarity and thus higher importance scores. The popular values distribution produced the lowest scores for Exact-Greedy and LSH-Greedy algorithms when running with larger datasets, which is expected because when generating popular values datasets we used Zipf’s law to make certain values appear very frequently across tuples. Since many tuples share these common values, they are more similar to each other, this reduces their average dissimilarity and lower their importance score.

The runtime behavior reveals an interesting pattern. For Exact-Enumeration and Exact-Greedy, runtime remains relatively consistent across distributions. This is because both algorithms compute distances for all pairs regardless of data characteristics. However, LSH-Greedy shows significant runtime variation, it took 29.05 seconds for IID-uniform, 64.79 seconds for clustered, and 208.28 seconds for popular values. This $7\times$ difference is related to how LSH works. LSH uses hashing to identify similar pairs within a threshold. In the popular values distribution, many tuples share the same frequently-occurring values, making them similar and causing LSH to identify and store many more near-neighbor pairs. This increases both memory usage and computation time during the precomputation phase. In contrast, IID-uniform data has more evenly distributed values, resulting in fewer LSH matches and faster execution. The clustered distribution falls between these extremes because while clusters have internal similarity, different clusters are quite dissimilar from each other.

3.3 Method 3 Results

Method 3 implements an exact Shapley value-based residual importance calculation, it requires evaluating all possible subsets of the data to determine each tuple’s contribution to diversity of these subsets. The algorithm has complexity $O(2^n \times n \times D)$, where the 2^n term comes from the number of subsets, the n term from computing contributions for each tuple, and D from distance calculations. This exponential complexity in the number of rows severely limits scalability, making method 3 feasible only for very small datasets.

We first evaluated method 3 on the IID-uniform distribution with varying dataset sizes from 10 rows to 25 rows. Table 8 shows the results. The runtime grows exponentially: from 17.28 seconds at 10 rows to 31.24 seconds at 15 rows ($1.8\times$ increase), then to 544.14 seconds at 20 rows ($17.4\times$ increase), and finally to 20,081.19 seconds (5.6 hours) at 25 rows ($36.9\times$ increase). This acceleration in growth rate is characteristic of exponential complexity. The number of subsets grows from $2^{10} = 1,024$ to $2^{15} = 32,768$ to $2^{20} = 1,048,576$ to $2^{25} = 33,554,432$. Even with our batch processing and Spark parallelization, evaluating over 33 million subsets takes hours. Attempting to run Method 3 on datasets larger than 25 rows becomes completely impractical, as doubling to 50 rows would require evaluating over 10^{15} subsets.

Table 8: Method 3 Runtime Performance on IID-Uniform Distribution

Rows (n)	Threshold (T)	Diversity score	Runtime (sec)
5	2	0.66	8.61
10	5	0.88	17.28
15	5	0.877	31.24
20	8	0.923	544.14
25	8	0.931	20,081.19

To understand how different data distributions affect method 3’s performance, we tested all three distributions (IID-uniform, clustered, and popular values) at various dataset sizes. Table 9 presents the complete results. The diversity scores show some variation across distributions, we observed that popular values consistently achieves slightly lower scores compared to IID-uniform and clustered when we test them on the same datasets. This pattern is consistent with what we observed in method 2 because popular values creates more similar tuples due to frequently-occurring values, reducing overall diversity. The clustered and IID-uniform distributions produce comparable scores, with clustered occasionally achieving slightly higher diversity when the selected tuples come from different clusters.

Table 9: Method 3 Performance Across Distributions

Distribution	Rows (n)	Threshold (T)	Score	Runtime (s)
iid_uniform	5	2	0.66	8.61
iid_uniform	10	5	0.88	17.28
iid_uniform	15	5	0.877	31.24
iid_uniform	20	8	0.923	544.14
iid_uniform	25	8	0.931	20,081.19
clustered	5	2	0.611	5.11
clustered	10	5	0.883	16.74
clustered	15	5	0.878	32.21
clustered	20	8	0.927	498.68
clustered	25	8	0.865	17,967.87
popular_values	5	2	0.611	6.09
popular_values	10	5	0.770	18.23
popular_values	15	5	0.792	34.11
popular_values	20	8	0.833	484.63
popular_values	25	8	0.846	18,727.16

The runtime differences across distributions are relatively minor compared to the overwhelming impact of dataset size. The similarity in runtimes makes sense because method 3’s complexity is

fundamentally determined by the number of subsets (2^n), which depends only on dataset size, not on the data values themselves. Minor variations likely come from differences in how quickly centroids converge or how efficiently the incremental centroid update performs with different value distributions, but these are second-order effects.

The key takeaway from the tests with method 3 is that method 3's exponential complexity makes it fundamentally unscalable beyond toy datasets. Even with optimizations like batch processing, Spark parallelization, incremental centroid updates, and precomputed factorials, we cannot overcome the fact that the number of subsets doubles with every additional row. To use Shapley value-based importance on larger datasets would require approximations, such as sampling a subset of all possible subsets rather than evaluating all of them, which is also an active area of research.

4 CONCLUSION

In this project, we implemented three methods to find the most important subset of the dataset, addressing the problem of companies running out of storage space and needing to remove data while keeping only the truly most valuable information. Out of these approaches, method 1 produced the best results in terms of runtime and scalability, reaching up to hundreds of thousands of rows. In contrast, method 2 and method 3 showed limited scalability, with subset enumeration of both methods feasible only for tens of rows, but method 2's approximation versions scaled to tens of thousands of rows. This difference comes from their importance calculations: method 1 relies solely on popularity, while method 2 and method 3 incorporate diversity and residual importance to find the most important subset. We can observe a trade-off between the complexity of importance calculation and scalability.

However, the question arises is whether that trade-off is worth it. These methods are intended to be used by organizations with large-scale data, which means they have to have good scalability. Yet, if a method's calculation does not accurately capture what is truly valuable because of its simplicity, then there is a risk of not keeping essential data. So, both scalability and accuracy are important for the organizations. Our experiments show that the choice of method depends on the specific needs. For scenarios where query coverage is the main goal and datasets are large, method 1 is the most practical solution. When both popularity and diversity matter and moderate dataset sizes are acceptable, method 2's approximation versions offer a good balance between quality and efficiency. method 3 is valuable despite its computational cost to find the diverse subset from a small dataset. In the end, organizations that want to remove data must consider their data characteristics, scalability needs, and the risks of losing potentially valuable information when deciding which approach to apply.

ACKNOWLEDGMENTS

We would like to thank our teacher Prof Yannis Velegrakis for his lectures and insights on working with big data.

REFERENCES

- [1] Daniel Cozzolino. 2025. The contribution of digital and sensing technologies and big data towards sustainable food supply and value chains. *Sustainable Food Technology* 3, 1 (2025), 181–187.
- [2] Tala Talaei Khoei and Aditi Singh. 2025. Data reduction in big data: a survey of methods, challenges and future directions. *International Journal of Data Science and Analytics* 20, 3 (2025), 1643–1682.
- [3] Peyman Passban, Mehdi Rezagholizadeh, and Andy Way. 2025. Introduction and Fundamentals. In *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques*. Springer, 3–16.
- [4] Md Jawadur Rahim, Ahlina Afroz, and Omolola Akinola. 2025. Predictive analytics in healthcare: Big data, better decisions. *International Journal of Scientific Research and Modern Technology* 4, 1 (2025), 1–21.
- [5] Andy Way, Peyman Passban, and Mehdi Rezagholizadeh. 2025. Remaining Issues for AI. *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques* 7 (2025), 179.